Karadeniz Teknik Üniversitesi

Fen Fakültesi – Bilgisayar Bilimleri Bölümü

Çok Kanallı Programlamaya Giriş

Bölüm 9 – Hafta 9

Yazılım Geliştirme II (BILB2000)

Bahar Dönemi – 9. Hafta

Bu Haftanın Planı

1. Ünite 1: Eşzamanlılık Kavramları ve OpenMP
2. Ünite 2: std::thread Temelleri
3. Ünite 3: std::mutex ve Senkronizasyon
4. Ünite 4: Atomik İşlemler ve OBYS Entegrasyonu
5. Özet

Hafta 9'un Büyük Resmi

Neden Çok Kanallı Programlama?

Bölüm 1–8 boyunca OBYS'yi STL ve Qt ile inşa ettik. Ancak büyük dosya kaydederken, toplu rapor üretirken ya da grafik canlandırırken uygulama *donuyor*. Bu hafta bu sorunu kökünden çözüyoruz.

Kazanımlar

- OpenMP ile ilk paralellik
- `std::thread` ile izlek yönetimi
- Mutex ve senkronizasyon
- Atomik işlemler

Altın Kural

"Concurrency is not parallelism."
– Rob Pike

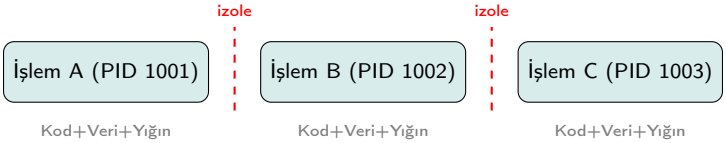
Eşzamanlılık yapı, paralellik yürütmedir.

Ünite 1: Eşzamanlılık Kavramları ve OpenMP

İşlem (Process) Nedir?

Tanım

İşlem, çalışan bir programın bellekteki örneğidir. Her işlemin kendine ait **adres alanı**, **dosya tanıtıcıları** ve **yığın (stack)** bölgesi vardır.



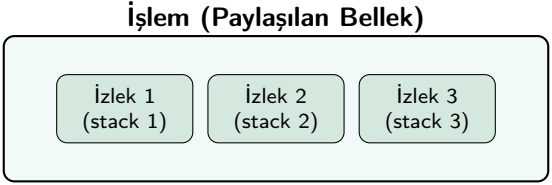
IPC

İşlemler arası iletişim pahalıdır: paylaşılan bellek, soket, pipe gibi mekanizmalar gerektirir.

İzlek (Thread) Nedir?

Tanım

İzlek, bir işlem içinde çalışan hafif yürütme birimidir. Aynı işlemin izlekleri **heap**, **kod** ve **global veri** alanını paylaşır; her birinin kendine ait **yığını (stack)** vardır.



Heap, Kod Segmenti, Global Değişkenler – **PAYLAŞIMLI**

İşlem vs İzlek Karşılaştırması

Özellik	İşlem (Process)	İzlek (Thread)
Bellek alanı	Ayrı	Paylaşımlı
Oluşturma maliyeti	Yüksek	Düşük
Bağlam değiştirme	Yavaş	Hızlı
İletişim	IPC gerekir	Doğrudan bellek
Hata izolasyonu	Güçlü	Zayıf

OBYS Bağlamı

OBYS'nin raporlama motorunu ayrı bir *izlekte* çalıştırmak, ayrı bir *işlem* başlatmaktan çok daha verimli – çünkü izlek, öğrenci verilerine doğrudan erişebilir.

Eşzamanlılık vs Paralellik

Eşzamanlılık (Concurrency)

Birden fazla görevin *zaman dilimlerini paylaşarak* ilerlediği **yapısal** bir kavram.

Tek çekirdekte bile olabilir.

Tek çekirdek – zaman paylaşımı

Paralellik (Parallelism)

Birden fazla görevin *aynı anda* farklı çekirdeklerde yürütüldüğü **fiziksel** bir kavram.

İki çekirdek – gerçek paralel

OpenMP Nedir?

Tanım

OpenMP (Open Multi-Processing), C/C++ ve Fortran programlarında **pragma yönergeleriyle** paralellik ekleyen bir API'dir.

Avantajları

- Tek satır eklemeyle paralellik
- Mevcut seri kodu bozmaz
- Derleyici desteği yaygın
- Döngü bazlı paralelllikte ideal

Sınırları

- Yalnızca paylaşılan bellek
- İnce taneli kontrol zor
- Platform bağımlılıkları
- Karmaşık senkronizasyonda yetersiz

İlk OpenMP Programı: parallel for

```
#include <omp.h>
#include <iostream>
#include <vector>

int main() {
    std::vector<double> notlar(10000, 75.0);
    double toplam = 0.0;

    #pragma omp parallel for reduction(+:toplam)
    for (int i = 0; i < static_cast<int>(notlar.size()); ++i)
        toplam += notlar[i];

    std::cout << "Ortalama: " << toplam / notlar.size() << '\n';
}
```

Derleme

```
g++ -std=c++20 -fopenmp -o ornek ornek.cpp
```

OpenMP: reduction ve critical

reduction Yönergesi

Her izlek kendi yerel kopyasıyla çalışır, sonunda sonuçlar birleştirilir.

```
double toplam = 0.0;
#pragma omp parallel for \
    reduction(+:toplam)
for (int i = 0; i < n; ++i)
    toplam += veri[i];
```

critical Yönergesi

Kritik bölgeye aynı anda yalnızca bir izlek girebilir.

```
int enYuksek = 0;
#pragma omp parallel for
for (int i = 0; i < n; ++i) {
    #pragma omp critical
    {
        if (veri[i] > enYuksek)
            enYuksek = veri[i];
    }
}
```

Dikkat

`critical` bloğu seri hale getirir – aşırı kullanımı performansı öldürür. Mümkünse `reduction` tercih et.

OpenMP ile OBYS: Toplu Not Ortalaması

```
// 100.000 ogrencinin not ortalamasini hesapla
std::vector<Ogrenci> ogrenciler = veriYukle();
std::vector<double> ortalamalar(ogrenciler.size());

#pragma omp parallel for
for (int i = 0; i < static_cast<int>(ogrenciler.size()); ++i) {
    double toplam = 0.0;
    for (const auto& not_ : ogrenciler[i].notlar)
        toplam += not_.deger;
    ortalamalar[i] = toplam / ogrenciler[i].notlar.size();
}
```

Sonuç

Herbir öğrencinin not ortalaması bağımsız hesaplanabilir → **mükemmel paralellik** (embarrassingly parallel).

OpenMP Yetmediğinde

OpenMP Nerede Yetersiz Kalır?

1. Döngü dışı paralel görevler (ör. GUI + arka plan kaydetme)
2. İnce taneli senkronizasyon (üretici-tüketici deseni)
3. Farklı yaşam sürelerine sahip izlek yönetimi
4. Platform bağımsız taşınabilirlik (derleyici bayrağı gerekir)
5. Asenkron sonuç bekleme (`future/promise`)

Motivasyon: std::thread

C++11 ile gelen `std::thread`, `std::mutex`, `std::atomic` ve `std::future` ailesi tüm bu ihtiyaçları standart kütüphane düzeyinde karşılar.

Artık derleyiciye özel pragma'lara bağımlı değiliz!

Eşzamanlılığın Zorlukları

Data Race	Deadlock
Livelock	Starvation
False Sharing	Memory Order

Kötü Haber

Bu sorunlar genellikle *deterministik değildir* – test sırasında ortaya çıkmayıp üretimde patlayabilir.

İyi Haber

Modern C++ araçları (mutex, lock_guard, atomic, scoped_lock) bu sorunları **sistematik** olarak çözmek için tasarlanmıştır.

Ünite 1 Özet

Öğrendiklerimiz

- İşlem → izole bellek, ağır; izlek → paylaşımlı bellek, hafif
- Eşzamanlılık = yapı (tek çekirdekte bile); paralellik = yürütme (çok çekirdek)
- OpenMP: `#pragma omp parallel for`, `reduction`, `critical`
- OpenMP basit döngülerde harika; karmaşık senaryolarda `std::thread` gerekir

Ünite 2: std::thread Temelleri

std::thread: C++ Standart İzlek Sınıfı

Neden std::thread?

C++11 ile gelen `std::thread`, işletim sistemi izlek API'lerini (`pthread`, `CreateThread`) tek bir taşınabilir arayüz altında birleştirir.

Başlık Dosyası

```
#include <thread>
```

Derleme

```
g++ -std=c++20 -pthread ...
```

Temel İlke

`std::thread` nesnesi oluşturulduğu anda izlek çalışmaya başlar. Nesne yok edilmeden önce **mutlaka** `join()` veya `detach()` çağrılmalıdır, aksi halde `std::terminate`!

Yol 1: Fonksiyon İşaretçisi

```
#include <iostream>
#include <thread>

void selamla(int id) {
    std::cout << "Merhaba, ben izlek " << id << '\n';
}
int main() {
    std::thread t1(selamla, 1);
    std::thread t2(selamla, 2);
    t1.join();  // t1 bitene kadar bekle
    t2.join();  // t2 bitene kadar bekle
}
```

Çıktı (sıra değişebilir)

```
Merhaba, ben izlek 1
Merhaba, ben izlek 2
```

Yol 2: Lambda İfadesi

```
#include <iostream>
#include <thread>

int main() {
    int deger = 42;
    std::thread t([deger]() {
        std::cout << "Lambda izlek, deger = "
                  << deger << '\n';
    });
    t.join();
    return 0;
}
```

Neden Lambda?

- Kısa işler için ayrı fonksiyon yazmaya gerek kalmaz
- Yakalama listesi ile bağlam taşınır
- Modern C++ projelerde en sık tercih edilen yöntem

Yol 3: Functor (Çağrılabilir Nesne)

```
struct NotHesaplayici {
    double toplam = 0.0;
    int    sayi   = 0;

    void operator()(const std::vector<double>& notlar) {
        for (double n : notlar) toplam += n;
        sayi = static_cast<int>(notlar.size());
    }
};

int main() {
    std::vector<double> notlar = {85.0, 92.0, 78.0, 95.0};
    NotHesaplayici hesap;

    std::thread t(std::ref(hesap), std::cref(notlar));
    t.join();

    std::cout << "Ortalama: " << hesap.toplam / hesap.sayi << '\n';
}
```

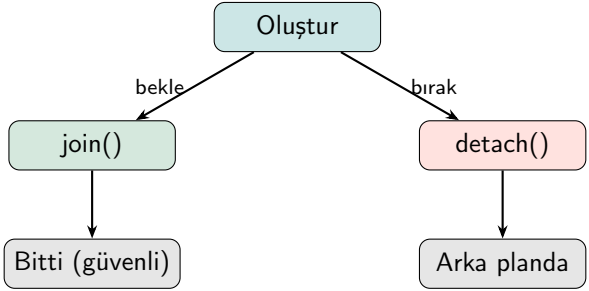
Thread Oluşturma: Üç Yol Karşılaştırması

Özellik	Fonksiyon	Lambda	Functor
Kod kısalığı	Orta	Yüksek	Düşük
Durum taşıma	Parametreler	Yakalama	Üye değişkenler
Yeniden kullanım	Kolay	Zor	Kolay
Okunabilirlik	İyi	İyi	Orta

Tavsiye

Kısa, tek seferlik işler → **lambda**. Tekrar kullanılacak mantık → **fonksiyon**. Durum tutması gereken iş nesnesi → **functor**.

join() ve detach(): İzlek Yaşam Döngüsü



join()

Çağıran izlek, hedef izlek bitene kadar **bloke olur**. En güvenli yöntem.

detach()

İzlek arka plana gönderilir, ana izlekle bağı kopar. Yaşam süresi denetimi **sizin sorumluluğunuz**.

join() Pratikte

```
void uzunIslem() {
    // 2 saniye suren bir hesaplama
    std::this_thread::sleep_for(std::chrono::seconds(2));
    std::cout << "Islem tamamlandi!\n";
}

int main() {
    std::thread t(uzunIslem);

    std::cout << "Ana izlek: bekliyorum...\n";
    t.join();  // t bitene kadar burada durur
    std::cout << "Ana izlek: devam ediyorum.\n";
}
```

Çıktı (garanti sıra)

```
Ana izlek: bekliyorum...
Islem tamamlandi!
Ana izlek: devam ediyorum.
```

detach() ve Tehlikeleri

```
void tehlikeliIs(int& veri) {
    std::this_thread::sleep_for(std::chrono::seconds(1));
    std::cout << veri << '\n';  // veri hala yasiyormu?
}

int main() {
    int x = 42;
    std::thread t(tehlikeliIs, std::ref(x));
    t.detach();
    // main biter, x yok edilir -> TANIMSIZ DAVRANIS!
}
```

Uyarı

`detach()` kullandığında, izleğin eriştiği tüm verilerin izleğin ömründen **daha uzun yaşadığından** emin ol! Genelde `join()` tercih edilmelidir.

joinable() ile Güvenli Yönetim

```
void guvenliTemizlik(std::thread& t) {
    if (t.joinable()) {
        t.join();
    }
}

int main() {
    std::thread t([]{ /* is yap */ });

    // ... istisna olabilir ...

    guvenliTemizlik(t);
}
```

Kural

`std::thread` yıkıcısı çağrıldığında izlek hala `joinable()` ise program `std::terminate` ile çöker. Her zaman kontrol et!

Parametre Geçirme: Değer (Copy)

```
void isle(std::string mesaj) {
    std::cout << mesaj << '\n';
}
int main() {
    std::string s = "Merhaba";
    std::thread t(isle, s);  // s kopyalanir
    t.join();
    // s hala "Merhaba" -- degismez
}
```

Varsayılan Davranış

`std::thread` yapıcısı argümanları **varsayılan olarak kopyalar**. Bu güvenlidir, ancak büyük nesneler için maliyetli olabilir.

Parametre Geçirme: Referans (std::ref)

```
void ortalamaHesapla(const std::vector<double>& notlar,
                     double& sonuc) {
    double toplam = 0.0;
    for (double n : notlar) toplam += n;
    sonuc = toplam / notlar.size();
}
int main() {
    std::vector<double> notlar = {85, 92, 78, 95};
    double ort = 0.0;
    std::thread t(ortalamaHesapla,
                  std::cref(notlar),   // const referans
                  std::ref(ort));      // degistirilebilir ref
    t.join();
    std::cout << "Ortalama: " << ort << '\n';  // 87.5
}
```

Önemli

`std::ref` / `std::cref` kullanmazsanız, thread argümanı **her zaman kopyalanır** – referans parametresi bile olsa!

Parametre Geçirme: Taşıma (std::move)

```
void dosyaIsle(std::unique_ptr<DosyaVerisi> veri) {
    std::cout << "Dosya boyutu: " << veri->boyut << '\n';
}

int main() {
    auto veri = std::make_unique<DosyaVerisi>("rapor.obys");

    std::thread t(dosyaIsle, std::move(veri));
    t.join();
    // veri artik nullptr -- sahiplik aktarildi
}
```

Ne Zaman Taşıma?

- Kopyalanamayan tipler: `unique_ptr`, `std::thread`
- Büyük nesneleri kopyalamak yerine sahiplik aktarmak istediğinde
- Kaynağın tek sahibi izlek olacaksa → en temiz çözüm

Parametre Geçirme Özeti

Değer
Güvenli, kopya maliyetli

std::ref
Hızlı, paylaşım data race riski

std::move
Sahiplik aktarımı en temiz yol

Pratik Kural

1. Küçük/basit veri → değer ile kopyala
2. Büyük salt-okunur veri → `std::cref`
3. Sonucu geri almak istiyorsan → `std::ref` + senkronizasyon
4. Sahiplik devri → `std::move`

Ünite 2 Özet

Öğrendiklerimiz

- `std::thread` – fonksiyon, lambda, functor ile oluşturma
- `join()` = bekle ve birleştir; `detach()` = arka plana bırak
- `joinable()` kontrolü yapılmazsa → `std::terminate`
- Parametre: varsayılan kopya, `std::ref` ile referans, `std::move` ile taşıma

Bir Sonraki Adım

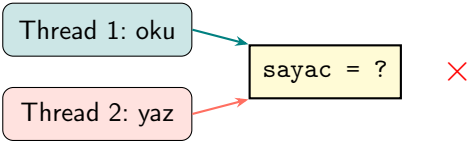
Birden fazla izlek aynı veriye eriştiğinde ne olur? → Ünite 3: Mutex ve Senkronizasyon

Ünite 3: std::mutex ve Senkronizasyon

Data Race: Sessiz Katil

Tanım

İki veya daha fazla izlek **aynı bellek konumuna** eşzamanlı erişir ve en az biri **yazma** yapıyorsa → **data race** (tanımsız davranış!).



OBYS Senaryosu

İki izlek aynı anda `toplamNot` değişkenine yazarsa, kayıp güncelleme (lost update) oluşur: 85 + 92 yerine yalnızca 92 kalabilir!

Data Race Pratikte

```
#include <thread>
#include <iostream>

int sayac = 0;
void artir(int n) {
    for (int i = 0; i < n; ++i)
        ++sayac;  // DATA RACE!
}
int main() {
    std::thread t1(artir, 1000000);
    std::thread t2(artir, 1000000);
    t1.join();
    t2.join();
    std::cout << "Sayac: " << sayac << '\n';
    // Beklenen: 2000000, Gercek: 1534821 (her calismada farkli!)
}
```

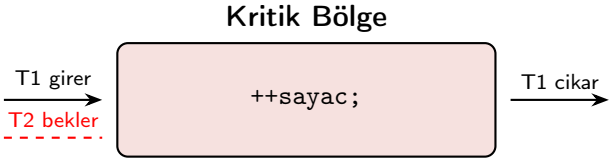
Sorun

++sayac atomik bir işlem **değildir**: oku → artır → yaz. İzlekler araya girebilir!

Kritik Bölge (Critical Section)

Tanım

Paylaşılan veriye erişen kod bölgesi **kritik bölge**dir. Aynı anda yalnızca **bir** izlek bu bölgede olmalıdır.



Çözüm Araçları

`std::mutex`, `std::lock_guard`, `std::unique_lock`, `std::scoped_lock`

std::mutex: Karşılıklı Dışlama

```
#include <mutex>
#include <thread>

int sayac = 0;
std::mutex mtx;
void artir(int n) {
    for (int i = 0; i < n; ++i) {
        mtx.lock();
        ++sayac;     // artik guvenli
        mtx.unlock();
    }
}
```

Sorun: Manuel lock/unlock

- İstisna fırlarsa `unlock()` atlanır → **kilitlenme (deadlock)**
- Her yolda unlock çağırmak hatalara açık
- RAII tercih et: `lock_guard`

std::lock_guard: RAII Kilit Yönetimi

```
void artir(int n) {
    for (int i = 0; i < n; ++i) {
        std::lock_guard<std::mutex> kilit(mtx);
        ++sayac;
    }  // kilit nesnesi yok olur -> otomatik unlock
}
```

RAII Prensibi

Resource Acquisition Is Initialization: Yapıcıda kilitle, yıkıcıda aç. İstisna fırlasa bile kilit **garanti** açılır.

C++17 ile CTAD

```
std::lock_guard kilit(mtx);  // sablon parametresi otomatik
```

std::unique_lock: Esnek Kilit

lock_guard vs unique_lock

unique_lock her şeyi yapabilir; ek olarak:

- Ertelenmiş kilitleme (std::defer_lock)
- Zaman aşımlı kilitleme (try_lock_for)
- Kilidin sahipliğini taşıma (std::move)
- condition_variable ile kullanım (zorunlu)

```
std::unique_lock<std::mutex> kilit(mtx, std::defer_lock);
// ... bazi hazirliklar ...
kilit.lock();      // simdi kilitle
++sayac;
kilit.unlock();    // erken ac (gerekirse)
// ... daha fazla is ...
```

OBYS'de Mutex: Öğrenci Listesi Koruması

```
class OBYSServis {
    mutable std::mutex m_mtx;
    std::vector<Ogrenci> m_ogrenciler;
public:
    void ogrenciEkle(const Ogrenci& ogr) {
        std::lock_guard kilit(m_mtx);
        m_ogrenciler.push_back(ogr);
    }
    size_t ogrenciSayisi() const {
        std::lock_guard kilit(m_mtx);
        return m_ogrenciler.size();
    }
};
```

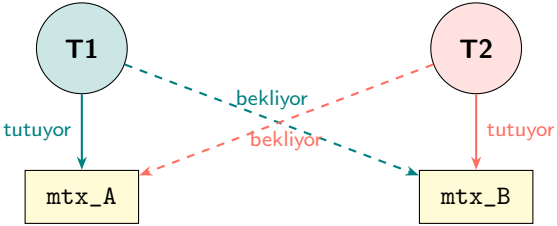
Neden mutable?

`ogrenciSayisi()` const bir metot; ancak mutex kilitleme mantıksal sabite aykırı değildir. `mutable` bu tutarsızlığı çözer.

Deadlock: Karşılıklı Kilitlenme

Tanım

İki veya daha fazla izlek, birbirinin tuttuğu kilidi beklediğinde hiçbiri ilerleyemez → **deadlock**.



Deadlock Örneği

```
std::mutex mtxA, mtxB;

void izlek1() {
    std::lock_guard kilitA(mtxA);    // A'yi kilitle
    std::this_thread::sleep_for(1ms);
    std::lock_guard kilitB(mtxB);    // B'yi bekle -> DEADLOCK
}

void izlek2() {
    std::lock_guard kilitB(mtxB);    // B'yi kilitle
    std::this_thread::sleep_for(1ms);
    std::lock_guard kilitA(mtxA);    // A'yi bekle -> DEADLOCK
}
```

Neden?

T1: A → B sırasıyla kilitlemeye çalışır. T2: B → A sırasıyla kilitlemeye çalışır. Sıra farklı → döngüsel bekleme → deadlock!

std::scoped_lock ile Deadlock Önleme

```
std::mutex mtxA, mtxB;

void izlek1() {
    std::scoped_lock kilit(mtxA, mtxB);  // ikisini birden
    // guvenli islem
}
void izlek2() {
    std::scoped_lock kilit(mtxA, mtxB);  // ayni sirada
    // guvenli islem
}
```

std::scoped_lock (C++17)

- Birden fazla mutex'i **atomik olarak** kilitler
- Sıra fark etmez – deadlock-free algoritma kullanır
- RAII: kapsam bitince otomatik açılır
- `lock_guard`'ın çok kilit için genelleştirilmiş hali

Deadlock Önleme: Altın Kurallar

1. **Sabit sıralama**: Tüm izlekler mutex'leri aynı sırayla kilitle
2. **scoped_lock**: Birden fazla mutex gerektiğinde tek seferde kilitle
3. **Kilitleme süresini kısalt**: Kritik bölgeyi mümkün olduğunca küçük tut
4. **try_lock**: Kilitleyemezsen geri çekil, tekrar dene
5. **Hiyerarşik kilit**: Mutex'lere seviye ata, düşük seviyeden yukarıya kilitle

Pratik Tavsiye

Şüphe durumunda `std::scoped_lock` kullan – derleyici doğru sıralamayı senin yerine halleder.

Ünite 3 Özet

Öğrendiklerimiz

- Data race = aynı veriye eşzamanlı yazma → tanımsız davranış
- `std::mutex`: `lock()`/`unlock()` – **asla** manuel kullanma
- `lock_guard`: RAII, basit, tek mutex
- `unique_lock`: esnek, ertelenmiş/zaman aşımlı, condition_variable
- `scoped_lock`: birden fazla mutex, deadlock-free
- Deadlock: döngüsel bekleme; çözüm = sabit sıra veya scoped_lock

Ünite 4: Atomik İşlemler ve OBYS Entegrasyonu

Atomik İşlem: Bölünemez Adım

Tanım

Atomik işlem, ara durumu **hiçbir zaman** dışarıdan gözlemlenemeyen, ya tamamen gerçekleşen ya da hiç gerçekleşmeyen işlemdir.

Mutex ile

1. Kilitle
2. Oku
3. Değiştir
4. Yaz
5. Kilit aç

→ 5 adım, ek maliyet

Atomik ile

1. Tek donanım talimatı ile oku-değiştir-yaz

→ Kilitsiz, hızlı

std::atomic: Temel Kullanım

```
#include <atomic>
#include <thread>
#include <iostream>

std::atomic<int> sayac{0};
void artir(int n) {
    for (int i = 0; i < n; ++i)
        ++sayac;  // atomik artırma -- data race YOK
}
int main() {
    std::thread t1(artir, 1000000);
    std::thread t2(artir, 1000000);
    t1.join(); t2.join();
    std::cout << "Sayac: " << sayac << '\n';  // her zaman 2000000
}
```

Karşılaştırma

Mutex ile korunan versiyon: kilit ek yükü. Atomik versiyon: donanım seviyesinde garanti, **mutex'siz**.

std::atomic Operasyonları

Operasyon	Açıklama
`load() / store(val)`	Değeri oku / yaz
`exchange(val)`	Eski değeri döndür, yenisini yaz
`fetch_add(val)`	Topla ve eski değeri döndür
`fetch_sub(val)`	Çıkar ve eski değeri döndür
`compare_exchange_strong`	CAS: karşılaştır ve değiştir
`compare_exchange_weak`	CAS: döngüde kullan

CAS (Compare-And-Swap)

```
int beklenen = 5;
sayac.compare_exchange_strong(beklenen, 10);
// Eger sayac == 5 ise -> sayac = 10, true dondur
// Eger sayac != 5 ise -> beklenen = sayac, false dondur
```

std::atomic_flag: En Basit Atomik Tip

```
std::atomic_flag bayrak = ATOMIC_FLAG_INIT;

void spinLock() {
    while (bayrak.test_and_set(std::memory_order_acquire))
        ;  // bayrak serbest olana kadar don
}

void spinUnlock() {
    bayrak.clear(std::memory_order_release);
}
```

Kullanım

- Lock-free garanti: her platformda atomik
- Basit bayrak senaryoları: iptal sinyali, hazır bildirimi
- Spin-lock implementasyonu (kısa bekleme süreleri)

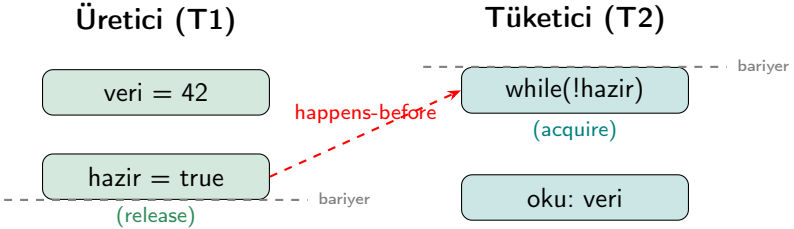
Bellek Sıralaması (Memory Order)

Sorun

Derleyici ve CPU, performans için bellek işlemlerini **yeniden sıralayabilir**. Tek izlekte sorun olmaz; çok izlekte veri tutarsızlığına yol açar.

Memory Order	Açıklama
`relaxed`	Sıralama garantisi yok; yalnızca atomiklik
`acquire`	Bu okumadan sonraki işlemler öne alınamaz
`release`	Bu yazmadan önceki işlemler sonraya atılamaz
`acq_rel`	Hem acquire hem release
`seq_cst`	Tam sıralama (varsayılan, en güvenli)

Acquire-Release Semantiği



Garanti

T1'deki `release` yazmadan **önce** yapılan tüm yazımlar, T2'deki `acquire` okumadan **sonra** görünür olur.

memory_order Pratikte

```
std::atomic<bool> hazir{false};
int veri = 0;

// Uretici izlek
void uretici() {
    veri = 42;                                          // normal yazma
    hazir.store(true, std::memory_order_release);  // bariyer
}
// Tuketici izlek
void tuketici() {
    while (!hazir.load(std::memory_order_acquire))  // bariyer
        ;
    assert(veri == 42);  // GARANTI: 42 gorulur
}
```

Neden seq_cst Kullanmıyoruz?

seq_cst her platformda en yavaşıdır. Üretici-tüketici deseninde acquire/release yeterli ve daha hızlıdır. Emin değilsen seq_cst kullan (varsayılan).

memory_order_relaxed: Ne Zaman?

```
std::atomic<int> islemSayisi{0};

void islemYap() {
    // ... bazi hesaplamalar ...
    islemSayisi.fetch_add(1, std::memory_order_relaxed);
}
```

Güvenli Kullanım

Yalnızca **sayaç** ve **istatistik** gibi, diğer değişkenlerle sıralama ilişkisi gerektirmeyen durumlarda `relaxed` kullanılabilir.

Tehlikeli Kullanım

Bir bayrak ile veri senkronize ediyorsan `relaxed` **asla** kullanma! → `acquire/release` veya `seq_cst` şart.

mutex vs atomic: Hangisini Seçmeliyim?

Kriter	mutex	atomic
Karmaşık veri yapısı	✓	×
Tek değişken	Gereksiz	✓
Birden fazla işlem	✓	CAS döngüsü
Performans	Orta	Yüksek
Kullanım kolaylığı	Kolay	Dikkat gerekir
Lock-free garanti	×	Platforma bağlı

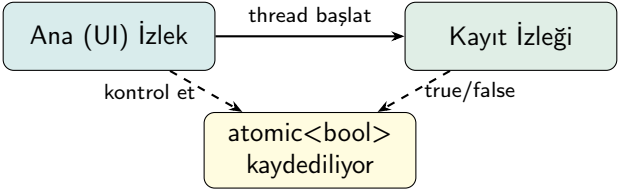
Pratik Kural

Tek bir sayaç veya bayrak → `atomic`. Birden fazla alan veya karmaşık yapı → `mutex`. İkisini birlikte de kullanabilirsin!

OBYS: Arka Planda Dosya Kaydetme

Problem

100.000 öğrencili OBYS dosyasını kaydederken ana pencere 3–5 saniye donuyor. Kullanıcı deneyimi kötü!



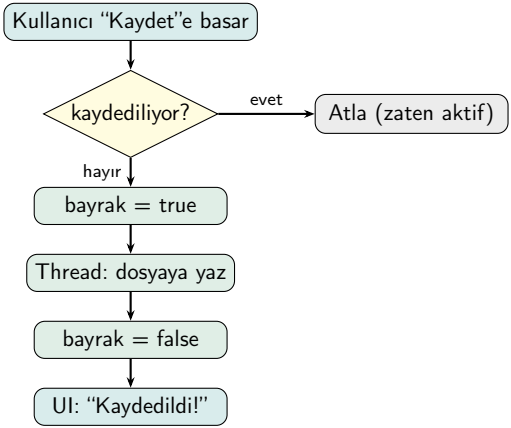
Çözüm

Kaydetme işlemini ayrı bir izlekte yap. `atomic<bool>` bayrağı ile durumu UI'a bildir.

OBYS Arka Plan Kaydetme: Kod

```
class OBYSServis {
    std::atomic<bool> m_kaydediliyor{false};
    std::mutex        m_veriMtx;
    std::vector<Ogrenci> m_ogrenciler;
public:
    void arkaPlandaKaydet(const std::string& dosyaYolu) {
        if (m_kaydediliyor.load()) return;  // zaten kaydediliyor
        m_kaydediliyor.store(true);
        std::thread([this, dosyaYolu]() {
            std::lock_guard kilit(m_veriMtx);
            dosyayaYaz(dosyaYolu, m_ogrenciler);
            m_kaydediliyor.store(false);
        }).detach();
    }
    bool kaydediliyorMu() const {
        return m_kaydediliyor.load();
    }
};
```

OBYS Kaydetme: İş Akışı



OBYS: Atomik İlerleme Bildirimi

```
class OBYSServis {
    std::atomic<int> m_ilerleme{0};  // 0-100

    void kaydetGoreviCalistir(/* ... */) {
        int toplam = m_ogrenciler.size();
        for (int i = 0; i < toplam; ++i) {
            kayitYaz(m_ogrenciler[i]);
            m_ilerleme.store((i + 1) * 100 / toplam,
                std::memory_order_relaxed);
        }
    }
};
// UI tarafında (ana izlek):
// int p = servis.ilerleme();  // relaxed okuma
// progressBar->setValue(p);
```

Neden `relaxed`?

İlerleme yüzdesi yaklaşık bir değerdir; tam sıralama gerekmez, `relaxed` yeterli ve hızlıdır.

Ünite 4 Özet

Öğrendiklerimiz

- `std::atomic`: kilitsiz, donanım destekli atomik işlemler
- `load()`, `store()`, `fetch_add()`, CAS operasyonları
- `memory_order`: relaxed, acquire, release, seq_cst
- Acquire-release = üretici-tüketici garantisi
- OBYS'de arka plan kaydetme: `atomic<bool>` bayrak + mutex veri koruması

Özet

Bölüm 9 Özet

Öğrendiklerimiz – 4 Büyük Adım

1. **Kavramlar + OpenMP**: İşlem vs izlek, eşzamanlılık vs paralellik, `#pragma omp parallel for`, reduction, critical
2. **std::thread**: Fonksiyon/lambda/functor ile oluşturma, join/detach, parametre geçirme (değer, ref, move)
3. **Mutex + Senkronizasyon**: Data race, lock_guard, unique_lock, scoped_lock, deadlock önleme
4. **Atomik İşlemler**: std::atomic, memory_order, acquire-release, OBYS arka plan kaydetme

OBYS Durumu

OBYS artık arka planda dosya kaydedebiliyor, UI donmuyor, ilerleme çubuğu atomik sayaçla güncelleniyor → **çok kanallı OBYS'nin temeli atıldı!**

Gelecek Hafta: Bölüm 10 – İleri Çok Kanallı Programlama

Neler Göreceğiz?

- `std::async` ve `std::future` ile asenkron görevler
- `std::condition_variable` ile üretici-tüketici deseni
- Thread pool tasarımı
- QThread ve Qt Concurrent ile GUI entegrasyonu

Uygulama Senaryosu

OBYS'de toplu transkript üretimi: her öğrenci bir görev, thread pool ile paralel işleme, sonuçlar `future` ile toplama, QProgressBar ile canlı ilerleme → responsive masaüstü uygulama!